Facultatea de Matematică și Informatică
Universitatea din București



Programare orientată pe obiecte

- suport de curs -

Anca Dobrovăț

An universitar 2025 – 2026
Semestrul I
Seria 26

Curs 2

Agenda cursului

- Recapitularea discuțiilor din cursul anterior (Generalități despre curs, Reguli de comportament, Diferențe cu C)

- Generalități despre OOP (Principiile programării orientate pe obiecte)

  – Clase, obiecte, modificatori de acces, funcții și clase prieten, constructori / destructor

1. C în C++

- Bjarne Stroustrup în 1979 la Bell Laboratories in Murray Hill, New Jersey
- 7 revizii: 1998 ANSI+ISO, 2003 (corrigendum), 2011 (C++11/0x), 2014, 2017 (C++ 17/1z), 2020, 2023
- Următoarea plănuită în 2026 (C++2c)
- Versiunea 1998: Standard C++, C++98
- C++ modern: ≥ C++11

1. C în C++

- C++98: a definit standardul inițial, toate chestiunile de limbaj, STL
- C++03: bugfix o unică chestie nouă: value initialization
- C++11: initializer lists, rvalue references, moving constructors, lambda functions, final, nullptr literal, sincronizare cu C99, etc.
- C++14: generic lambdas, binary literals, auto, variable template, etc.

1. C în C++

C++17:

- If constexpr()
- Inline variables
- Nested namespace definitions
- Class template argument deduction
- Hexadecimal literals
- etc

typename is permitted for template template parameter declarations (e.g., **template**<**template**<typename> **typename** X> **struct** …)

1. C în C++

- C++20:
  - Concepte: sintaxă mai "simplă" și erori mai "clare" la templates
  - Module
  - Spaceship operator <=> și operator==() = default

- C++23: <stacktrace>, <expected>

1. C în C++

- **Intrări și ieșiri**
  - Obiectele **cin** și **cout**, în plus față de funcțiile scanf și printf din limbajul C.
  - Nu necesită specificarea formatelor.
  - Pentru I/O type-safe cu format: <format> (C++20)
- **Comentarii pe o singură linie**
- **Citirea string-urilor până la primul caracter alb**

1. C în C++

Supraîncărcarea funcțiilor (un caz de *Polimorfism la compilare*)

Utilizarea mai multor funcții care au același nume

Identificarea se face prin numărul de parametri și tipul lor. **Tipul de întoarcere nu e suficient pentru a face diferența**

Simplicitate/corectitudine de cod

La compilare: transformat în nume unic prin name mangling

1. C în C++

```
#include <iostream>
using namespace std;

// abs2 is overloaded three ways
int abs2(int i);
double abs2(double d);
long abs2(long l);

int main() {
        cout << abs2(-10) << "\n";
        cout << abs2(-11.0) << "\n";
        cout << abs2(-9L) << "\n";
        return 0; }

int abs2(int i) {
        cout << "Using integer abs2()\n";
        return i<0 ? -i : i; }
```

```
double abs2(double d) {
        cout << "Using double abs2()\n";
        return d<0.0 ? -d : d; }

long abs2(long l)  {
        cout << "Using long abs2()\n";
        return l<0 ? -l : l;
}
```

```
Using integer abs2()
10
Using double abs2()
11
Using long abs2()
9
```

1. C în C++

Funcții cu valori implicite

Într-o funcție se pot declara valori implicite pentru unul sau mai mulți parametri.

La apel se poate omite specificarea valorii pentru acei parametri formali care au declarate valori implicite.

Argumentele cu valori implicite trebuie să fie amplasate la sfârșitul listei.

Valorile implicite se specifică o singură dată în definiție (de obicei în prototip).

1. C în C++

Funcții cu valori implicite

```
#include <iostream>
using namespace std;

void f (int a, int b = 12); // prototip cu menționarea valorii implicite pentru b

int main()
{
  f(1);
  f(1,20);
  return 0;
}

void f (int a, int b) {  cout<<a<<" - "<<b<<endl; }
```

1. C++

Tipul referință

O referință este, în esență, un pointer implicit, care acționează ca un alt nume al unui obiect (variabilă).

```
int i;
int *pi,j;
```

// int& ri; // eroare de sintaxă
int & ri=i; //ri este alt nume pentru variabila I

```
pi=&i;  // pi este adresa variabilei I
*pi=3;   // în zona adresată de pi se află valoarea 3
```

Pentru a putea fi folosită, o referință trebuie inițializată în momentul declarării, devenind un alias (un alt nume) al obiectului cu care a fost inițializată.

1. C++

Tipul referință

```
int a = 20;
int& ref = a;
cout<<a<<" "<<ref<<endl; // 20 20
```

ref++;

```
cout<<a<<" "<<ref<<endl; // 21 21
```

Obs: spre deosebire de pointeri care prin incrementare directă ajung să rețină o altă adresă, incrementarea referinței implică, de fapt, incrementarea valorii referite.

1. C++

Tipul referință

```
int a = 20;
int& ref = a;
cout<<a<<" "<<ref<<endl; // 20 20

int b = 50;
ref = b;

ref--;
cout<<a<<" "<<ref<<endl; // 49 49
```

Obs: nu puteți modifica obiectul spre care indică referința (intern, o referință este un pointer constant).

1. C++

Tipul referință

- o variabilă de tip referință trebuie să fie inițializată când este definită
- membrii unei clase de tip referință trebuie inițializați în constructori
- un parametru de funcție de tip referință se inițializează implicit la apel
- referințele nule sunt interzise intr-un program C++ valid.
  - putem întoarce referințe dangling (warning la compilare); // dangling – un pointer/referinta care “arata” catre o zona de memorie deja dezalocata
- nu se poate obține adresa unei referințe (&ref ne dă adresa variabilei)
  - intern sunt tot pointeri, dar așa e mai greu să îi "stricăm"
- se pot crea tablouri de referințe cu std::reference_wrapper

1. C vs C++

Transmiterea parametrilor

C

```
void f(int x)
    {   x = x *2;}

void g(int *x)
    {  *x = *x + 30;}

int main() {
  int x = 10;
  f(x);
  printf("x = %d\n”,x);
  g(&x);
  printf("x = %d\n“,x);
  return 0;
}
```

C++

```
void f(int x){   x = x *2;} //prin valoare
void g(int *x){  *x = *x + 30;} // prin pointer
void h(int &x){ x = x + 50;} //prin referință

int main()  {
  int x = 10;
  f(x);
  cout<<"x = "<<x<<endl;
  g(&x);
  cout<<"x = "<<x<<endl;
  h(x);
  cout<<"x = "<<x<<endl;
  return 0;
}
```

1. C vs C++

Transmiterea parametrilor

Observații generale

- parametrii formali - sunt creați la intrarea într-o funcție și distruși la retur;
- apel prin valoare - copiază valoarea unui argument într-un parametru formal ⇒ modificările parametrului nu au efect asupra argumentului;
- apel prin referință - în parametru este copiată adresa unui argument ⇒ modificările parametrului au efect asupra argumentului.
- apelurile de funcții, cu excepția celor de tip void, pot fi folosite ca operand în orice expresie validă.

1. C++

STL (Standard Template Library)

```
#include <vector>
using namespace std;
int main()
{
  vector<int> v1, v2{0, 10, 20};
  v1.push_back(20);
  v1.push_back(30);
  vector<int>::iterator i;
  for(i = v1.begin(); I != v1.end(); ++i)
      cout << *i << " ";
  for(auto elem : v2)
      cout << elem << " ";
  return 0;
}
```

2. Principiile programării orientate pe obiecte

Clasă

Obiect

Încapsulare

Modularitate

Agregare (compunere)

Moștenire (ierarhizare)

Polimorfism la compilare și polimorfism la execuție, etc.

2. Principiile programării orientate pe obiecte

Obiecte

- au stare și acțiuni (metode/funcții)
- au interfață (acțiuni) și o parte ascunsă (starea)
- Sunt grupate în clase, obiecte cu aceleași proprietăți
- Un **program orientat obiect** este o colecție de obiecte care interacționează unul cu celălalt prin mesaje (aplicând o metodă).

2. Principiile programării orientate pe obiecte

Clase

O **clasă** definește atribute și metode.

```
class X{
        //date membre
        //metode (funcții membre – funcții cu argument implicit obiectul curent)
        };
```

- menționează proprietățile generale ale obiectelor din clasa respectivă
- folositoare la încapsulare (ascunderea informației)
- reutilizare de cod: moștenire

2. Principiile programării orientate pe obiecte

Clase

- cu “class”
- obiectele instanțiază clase
- similare cu struct-uri
- au funcții
- specificatorii de acces: public, private, protected
- default: private
- protected: pentru moștenire, vorbim mai târziu

2. Principiile programării orientate pe obiecte

Clase

```
class nume_clasă {
        private variabile și funcții membru
specificator_de_acces:
        variabile și funcții membru
specificator_de_acces:
        variabile și funcții membru
// ...
specificator_de_acces:
        variabile și funcții membru
} listă_obiecte;
```

- putem trece de la public la private și iar la public, etc.

Clase

```
class employee {
// private din oficiu
        std::string name;
public:
// acestea sunt publice
        void set_name(std::string n);
        std::string get_name();
private:
// acum din nou private
        double wage;
public:
// înapoi la public
        void set_wage(double w);
        double get_wage();
};
```

```
class employee {
        std::string name;
        double wage;
public:
        void set_name(char *n);
        std::string get_name();
        void set_wage(double w);
        double get_wage();
};
```

Clase

- se folosește mai mult a doua variantă
- un membru (ne-static) al clasei poate avea inițializare în interiorul clasei
  - se recomandă inițializarea în acest fel pentru valori implicite și pentru a inițializa toți membrii de tipuri primitive
- nu putem avea ca membri obiecte de tipul clasei (putem avea pointeri/referințe la tipul clasei)

```
class employee {
    std::string name{"Employee"};
    double wage = 123.45;
    // employee manager;            // error
    employee* manager = nullptr; // ok
};
```

2. Principiile programării orientate pe obiecte

Clase

- variabilele de instanță (instance variables)

- membri de tip date ai clasei
  - în general private
  - pentru viteză se pot folosi “public” dar **NU LA ACEST CURS**

2. Principiile programării orientate pe obiecte

Clase

```
const int SIZE = 100;
// This creates the class stack.
class stack {
        int stck[SIZE];
        int tos;
public:
        void init();
        void push(int elem);
        int pop();
};
```

- init(), push(), pop() sunt funcții membru
- stck, tos: variabile membru

2. Principiile programării orientate pe obiecte

Clase

- se creează un tip nou de date
- un obiect instanțiază clasa
- funcțiile membru sunt date prin semnătură
- pentru definirea fiecărei funcții se folosește ::

stack mystack;

```
void stack::push(int elem)  {
        if(tos==SIZE) {
                cout << "Stack is full.\n";
        return; }
        stck[tos] = elem;
        tos++;
}
```

2. Principiile programării orientate pe obiecte

Clase

- :: scope resolution operator
- și alte clase pot folosi numele push() și pop()
- după instanțiere, pentru apelul push()

stack mystack;

- mystack.push(5);
- programul complet în continuare

Clase

```
#include <iostream>
using namespace std;
const int SIZE = 100;
// This creates the class stack.
class stack {
        int stck[SIZE];
        int tos;
public:
        void init();
        void push(int elem);
        int pop();
};
void stack::init()
{
        tos = 0;
}
void stack::push(int elem) {
        if(tos==SIZE) {
                cout << "Stack is full.\n";
                return;
        }
        stck[tos] = elem;
        tos++;
}
int stack::pop()
{
        if(tos==0) {
                cout << "Stack underflow.\n";
                return 0;
        }
        tos--;
        return stck[tos];
}
int main()
{
        stack stack1, stack2; // create two stack objects
        stack1.init();
        stack2.init();
        stack1.push(1);
        stack2.push(2);
        stack1.push(3);
        stack2.push(4);
        cout << stack1.pop() << " ";
        cout << stack1.pop() << " ";
        cout << stack2.pop() << " ";
        cout << stack2.pop() << "\n";
        return 0;
}
```

2. Principiile programării orientate pe obiecte

Încapsularea

- ascunderea de informații (data-hiding);
- separarea aspectelor externe ale unui obiect, care sunt accesibile altor obiecte, de aspectele interne ale obiectului care sunt ascunse celorlalte obiecte;
- definește apartenența unor proprietăți și metode față de un obiect;
- doar metodele proprii ale obiectului pot accesa starea acestuia.

2. Principiile programării orientate pe obiecte

Încapsularea (ascunderea informației)

```
class Test  {
  private:
    int x;
  public:
    void set_x(int a) {x = a;}
};

int main() {
  Test t;
  t.x = 34; // inaccesibil
  t.set_x(34); // ok
  return 0;
}
```

2. Principiile programării orientate pe obiecte

- **Agregarea** (ierarhia de obiecte) compunerea unui obiect din mai multe obiecte mai simple. (relație de tip “has a”)

- **Moștenirea** (ierarhia de clase) relație între clase în care o clasă moștenește structura și comportarea definită în una sau mai multe clase (relație de tip “is a” sau “is like a”);

2. Principiile programării orientate pe obiecte

Agregarea

```
class Profesor
{
    string nume;
    int vechime;
};

class Curs
{
    string denumire;
    Profesor profesor;
};
```

2. Principiile programării orientate pe obiecte

Moștenire

- multe obiecte au comportamente/proprietăți similare
- reutilizare de cod
- extinderea mai ușoară a codului

2. Principiile programării orientate pe obiecte

Moștenire

- terminologie
  - clasă de bază, clasă derivată
  - superclasă, subclasă
  - părinte, fiu

- mai târziu: funcții virtuale, identificare de tipuri în timpul rulării (RTTI)

2. Principiile programării orientate pe obiecte

Moștenire

- încorporarea componentelor unei clase în alta
- refolosire de cod
- detalii mai subtile pentru tipuri și subtipuri
- clasă de bază, clasă derivată
- clasa derivată conține toate elementele clasei de bază, mai adaugă noi elemente

2. Principiile programării orientate pe obiecte

```
class building {
        int rooms;
        int floors;
        int area;
public:
        void set_rooms(int num);
        int get_rooms(); // …
};

// house e derivată din building
class house : public building {
        int bedrooms;
        int baths;
public:
        void set_bedrooms(int num);
        int get_bedrooms();};
```

Moștenire

tip acces: public, private, protected

mai multe mai târziu

public: membrii publici ai building devin publici pentru house

2. Principiile programării orientate pe obiecte

Moștenire

- house NU are acces la membrii privați ai lui building
- așa se realizează încapsularea

- clasa derivată are acces la membrii publici ai clasei de bază și la toți membrii săi (publici și protected)

2. Principiile programării orientate pe obiecte

Polimorfism

- tot pentru claritate/ cod mai sigur
- Polimorfism la compilare: ex. max(int), max(float)
- Polimorfism la execuție: RTTI

2. Principiile programării orientate pe obiecte

Perspective

Curs 3

- Class, struct
- Funcții și clase prieten
- Funcții inline
- Constructori / destructor